Тема 5. Типы данных линейной структуры

Лекции 11, 12

Стеки, очереди, хеш-таблицы

Целью данной **темы** является изучение типов данных линейной структуры. В ней рассматриваются такие линейные структуры данных как списки, стеки, очереди и хеш-таблицы. Рассматриваются вопросы их программной реализации.

5.2. Стеки, деки

Стек – это последовательный список переменной длины, включение и исключение элементов из которого производится только с одной стороны. Стеки иногда называют магазинами. Для обозначения стеков часто используется аббревиатура LIFO (last-in/first-out – "последний вошел – первый вышел").

Стек используют для хранения элементов, доступных в вершине списка (*top*). Структура стека аналогична стопке тарелок на полке буфета или стопке книг. В этих примерах можно взять только верхний предмет, а добавить новый предмет можно, только положив его сверху.

В структуре стека важное место занимают операции добавления и удаления элементов. Операция *Push* добавляет элемент в вершину стека. Операция *Pop* удаляет элемент из стека. Рис. 5.13 иллюстрирует применение этих операций.

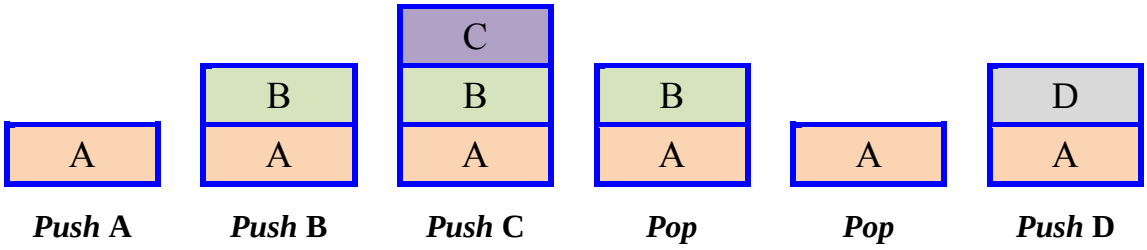


Рис. 5.13. Применение операций *Push* и *Pop*

Стек может быть реализован двумя способами: на основе массивов и при помощи указателей.

При реализации стека на основе одномерного массива размер стека ограничен максимальным количеством элементов массива. При этом может возникнуть проблема переполнения. На рис. 5.14 приведена схема реализации стека с помощью массива.

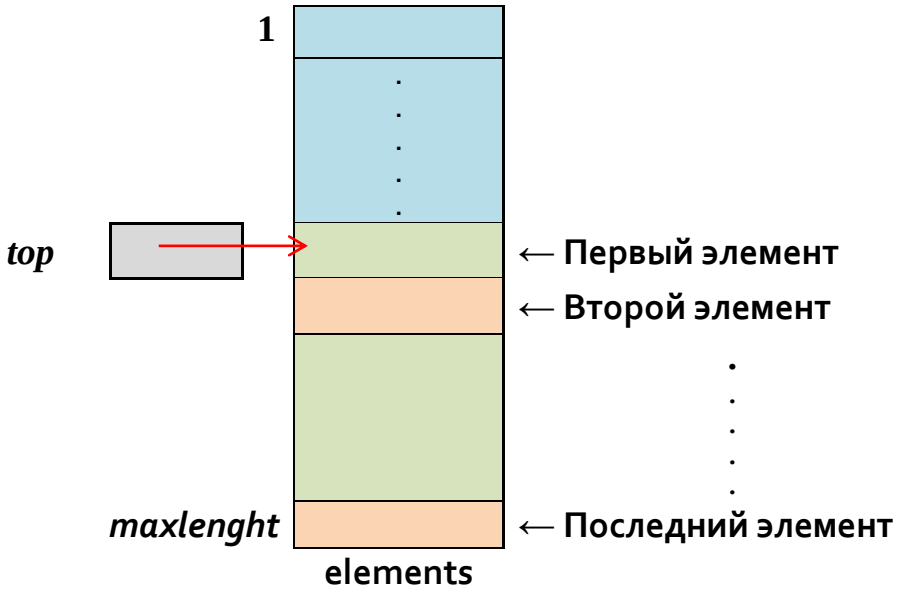


Рис. 5.14. Реализация стека на массиве

При реализации стека с помощью указателей размер стека ограничен доступным объемом свободной памяти. Графически структуру стека можно представить так, как показано на рис. 5.15.

Для стека характерен следующий набор операций:

- *Push* – добавление элемента в стек;
- *Pop* – удаление элемента из стека (с выдачей значения удаляемого элемента);
- выдача значения верхнего элемента;
- очистка стека;
- печать количества элементов стека.

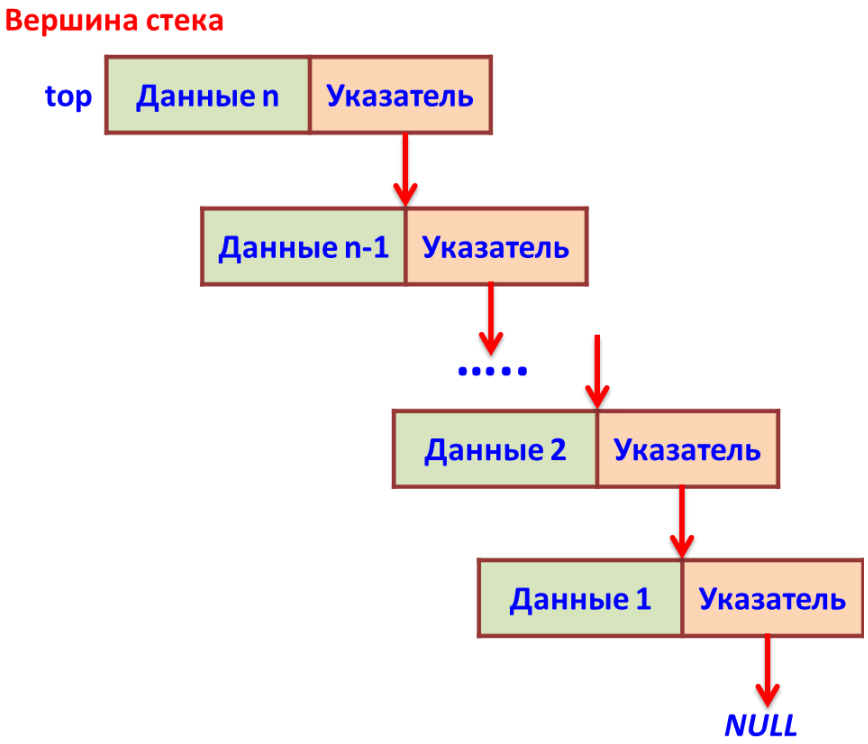


Рис. 5.15. Реализация стека при помощи указателей

Стек применяется:

- при работе с памятью, например, работа функций printf и scanf основана на использовании стека;
- для того, чтобы определить, является ли предложение палиндромом (палиндром – это строка, которая читается одинаково в прямом и в обратном направлениях);
- для вывода данных с различными основаниями;
- при разработке электронных калькуляторов.

Дек (*от англ. deq = double-ended-queue, «очередь с двумя концами»*) - структура данных с двусторонним доступом, хранящая упорядоченное множество значений, для которой определены следующие операции:

- очистить дек (сделать структуру пустой);
- проверить на пустоту;
- добавить элемент в начало;
- добавить элемент в конец;
- взять элемент из начала;
- взять элемент из конца.

5.3. Очереди

Очередь – это последовательный список переменной длины, в котором включение элементов производится с одного конца списка (с хвоста), а исключение элементов производится с другого конца (из головы). Очередь работает по принципу FIFO – First in First out (последний пришел, последний вышел). В качестве примера можно привести обслуживание клиентов в очереди.

При работе с очередью используются специальные указатели на начальную и конечную позиции. Эти указатели используются для вставки и удаления элементов из очереди. Начало очереди определяется первым элементом в очереди (*front*). Конец очереди – это место после его последнего элемента (*rear*).

Очередь может быть реализована двумя способами: на основе массивов и при помощи указателей.

Очередь, как и стек, не ограничивает количество элементов. Однако если для реализации очереди используется массив, может возникнуть условие полной очереди. Реализовать очередь на основе массивов можно двумя способами – линейной очередью и кольцевой очередью.

Рассмотрим простейшую линейную очередь на основе одномерного массива. На рис. 5.16 приведен пример добавления и удаления элементов в такую очередь.



Рис. 5.16. Добавление и удаление элементов в очередь, реализованную на массиве

При исключении элемента из очереди все оставшиеся элементы данных должны смещаться вперед на одну позицию. Данная модель не является эффективной. Например, очередь содержит 1000 элементов. Если один элемент удаляется из начала, тогда 999 элементов должны сместиться влево.

Кольцевая очередь является более эффективной. Элементы кольцевой очереди организованы логически в окружность. Переменная front всегда является местоположением первого элемента в очереди, и она продвигается вправо по кругу по мере выполнения удалений. Рис. 5.17 иллюстрирует процесс добавления и удаления элементов в кольцевой очереди.

При реализации очереди с помощью указателей размер очереди ограничен доступным объемом свободной памяти. Графически очередь можно представить так, как показано на рис. 5.18.

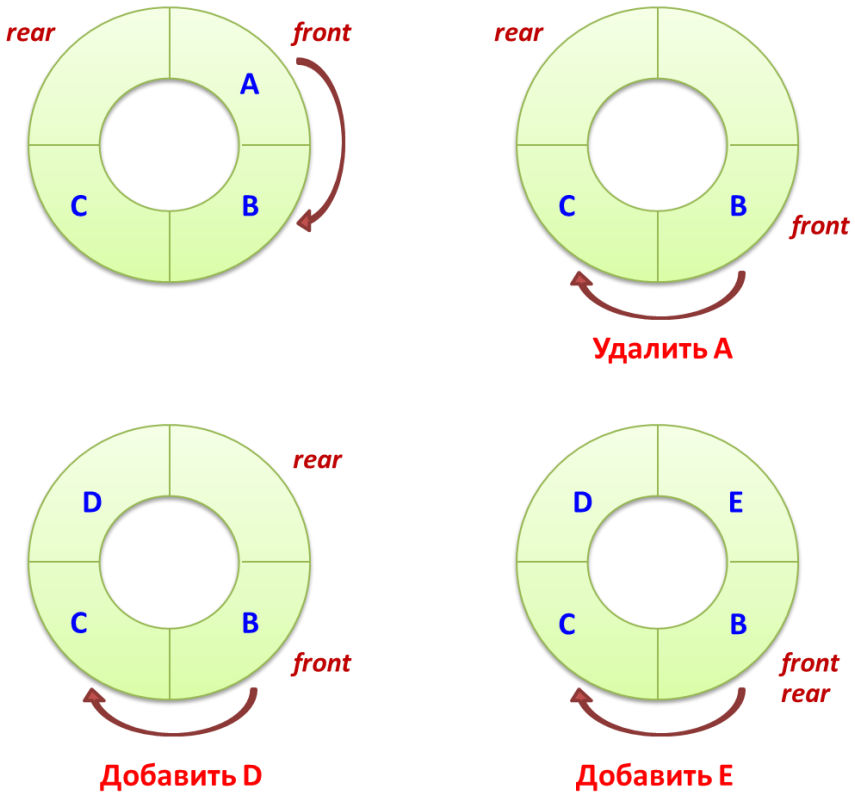


Рис. 5.17. Добавление и удаление элементов в кольцевой очереди

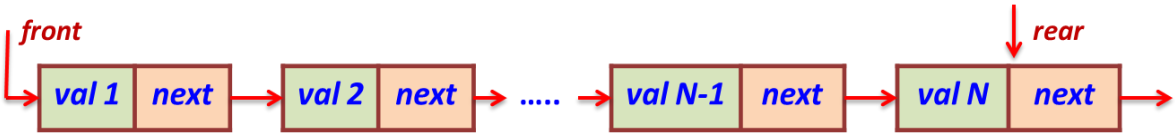


Рис. 5.18. Реализация очереди при помощи указателей

Для очереди характерен следующий набор операций:

- добавление элемента в очередь;
- удаление элемента из очереди (с выдачей значения исключаемого элемента);
- выдача значения первого элемента;
- очистка очереди;
- выдача количества элементов очереди.

Очередь применяется:

- в компьютерном моделировании (например, моделирование очереди клиентов в банке);
- в многопользовательских операционных системах;
- для сортировки данных.

Одной из разновидностей очереди является так называемая *очередь приоритетов* – это модифицированная версия очереди, в которой из списка удаляется элемент с высшим приоритетом.

Элементы в очереди приоритетов рассматриваются как пара ключ-значение, в которой ключ определяет уровень приоритета. Приоритет оценивается по некоторому внешнему критерию.

При удалении из очереди приоритетов элементов с одинаковым приоритетом сначала удаляется элемент, поступивший ранее.

Приоритет 0 определяется как высший (обычный приоритет имеет значение 20).

Рассмотрим следующий список элементов и их приоритетов (рис. 5.19):

Порядок хранения

Элемент №1	Элемент №2	Элемент №3	Элемент №4	Элемент №5
20	0	40	30	20

Рис. 5.19. Пример очереди приоритетов

Элементы из этой очереди приоритетов будут удаляться в порядке 2, 1, 5, 4, 3, что иллюстрирует рис. 5.20.

Порядок выполнения

Элемент №2	Элемент №1	Элемент №5	Элемент №4	Элемент №3
0	20	20	30	40

Рис. 5.20. Порядок удаления элементов из очереди приоритетов

Очередь приоритетов можно представить в виде нескольких очередей, где каждая очередь используется для своего приоритета:

Приоритет 1			
R_1	R_2	…	R_n
Приоритет 2			
O_1	O_2	…	O_n
Приоритет 3			
B_1	B_2	…	B_n

Элементы второй очереди обслуживаются только тогда, когда первая очередь пуста.

Очереди приоритетов используются в операционной системе, которая записывает процессы в список и затем выполняет их в порядке приоритетов.

5.4. Хеш-таблицы

Как было показано в предыдущей теме, большинство методов поиска основывается на упорядочении данных. Однако эффективность поиска можно повысить, если вместо упорядоченных структур использовать так называемые беспорядочные или перемешенные структуры данных. Для работы с такими структурами предназначены методы хеширования. Слово хеширование происходит от английского hash, означающего "крошить", "резать на кусочки".

Например, необходимо подвергнуть обработке много маленьких строк – *слов*, которые надо сохранять в некоторой единой структуре – *словаре*. Сами слова не требуют для своего представления сложной структуры: для их представления вполне достаточно стандартных способов описания строк. Однако для словаря необходимо выбрать такое представление, которое бы обеспечило бы максимальную эффективность выполнения основной операции над словарем – поиска слова в словаре. Основная идея применения функции расстановки или хэш-функции для решения данной задачи состоит в следующем. Словарь будем представлять в

виде массива слов, но помещать слова в него будем не в соответствии с алфавитным порядком, а в соответствии со значениями некоторой простой функции, вычисленной над словом. Такая функция – функция расстановки, – получая в качестве аргумента некоторое слово, выдает в качестве результата некоторое целое число – индекс в словаре, под которым следует хранить это слово. Если каждому слову будет соответствовать свое значение функции, то поиск в словаре становится ненужным. Вместо поиска осуществляется вычисление значения функции расстановки, после чего слово находится сразу же по вычисленному индексу.

Или предположим, что некоторая фирма выпускает детали и кодирует их семизначными цифрами. Для применения прямой индексации с использованием полного семизначного ключа потребовался бы массив из 100 млн. элементов. Ясно, что это привело бы к потере неприемлемо большого пространства, поскольку совершенно невероятно, что какая-либо фирма может иметь больше чем тысяча наименований изделий. Поэтому необходим некоторый метод преобразования ключа в какое-либо целое число внутри ограниченного диапазона. Тогда для хранения всего файла будет достаточно массива из 1000 элементов. Этот массив индексируется целым числом в диапазоне от 0 до 999 включительно. В качестве индекса записи об изделии в этом массиве используются три последние цифры номера изделия.

Методы хеширования основаны на *ассоциативной адресации*, идея которой состоит в том, чтобы сохранить без изменения неупорядоченную структуру и, имея ключ для поиска записи, определить адрес без предварительного сравнения ключей, то есть рассматривать адрес как функцию от ключа.

Организацию данных в виде таблицы назовем *хеш-таблицей*, если адрес каждой записи a этой таблицы определяется как значение некоторой функции $h(k)$, называемой *функцией расстановки* или *хеш-функцией*. Здесь k – значение ключа записи a.

Первое название функции отражает цель ее использования – вычисление некоторого индекса, с помощью которого можно расставить элементы (слова) в некотором множестве (например, словаре). Второе

название отражает метод, с помощью которого этот индекс вычисляется – разрезание элементов (слов) на части и перемешивание получившихся кусочков.

Если число записей невелико и заранее известно, то можно построить функцию преобразования заданного множества ключей в различные адреса (если возможно, то в последовательные адреса, т.к. это выгодно при использовании виртуальной памяти). Если же число записей велико, то найти такую функцию оказывается сложно. В случае если число различных значений ключей, вероятность появления которых отлична от нуля, превышает размер хеш-таблицы, построение функции оказывается невозможным. В этом случае приходится отказаться от идеи однозначности и рассмотреть хеш-функцию как функцию, рассеивающую множество ключей во множестве адресов.

Отказ от требования взаимно однозначного соответствия между ключом и адресом означает, что для двух различных ключей $k1 \neq k2$ значение хэш-функции может совпадать: $h(k1) = h(k2)$. К сожалению, на практике оказывается невозможным определить хэш-функцию так, чтобы каждому слову соответствовало свое уникальное значение индекса. Это приводит к тому, что два или более слов получают одно и то же значение хеш-функции. Говорят, что в этом случае происходит конфликт или *коллизия хеш-адресов*. Ключи $k1$ и $k2$ называются синонимами хэш-функции h, если $h(k1)=h(k2)$.

Для метода хеширования главными задачами являются выбор хэш-функции h и нахождение способа разрешения возникающих коллизий.

5.4.1. Универсальные функции расстановки

При выборе хеш-функции следует учитывать сложность ее вычисления, а также равномерность распределения значений, которая позволяет не только сократить число коллизий, но и не допустить скучивания значений в отдельных частях таблицы. Например, для обработки слов русского языка функция, выдающая по заданному слову номер его первой буквы в алфавите – это не очень удачный способ определения функции

расстановки. Во-первых, слова русского языка далеко не равномерно распределяются по первым буквам. Во-вторых, к этой неравномерности добавляется еще и упорядоченность, присущая алгоритмам обработки текстов. В такой ситуации лучше всего было бы, чтобы даже незначительно отличающиеся друг от друга слова имели бы существенно различные значения хеш-функции.

Для каждого конкретного множества возможных ключей можно изобрести (подобрать, найти) свою, возможно наилучшую, хеш-функцию распределения ключей по таблице. Но существуют и универсальные хеш-функции, дающие хорошие результаты в большинстве случаев. Рассмотрим некоторые из них.

Метод деления. В методе деления в качестве значения хеш-функции h используется остаток от деления ключа на некоторое целое число M:

$$h(k)=k \bmod m,$$

где m обычно равняется размеру хэш-таблицы. Эффективность рассеивания ключей во многом зависит от значения m. Не стоит выбирать m равным степени основания системы счисления, так как значением хэш-функции будут просто младшие разряды ключа. Например, для символьных ключей не следует выбирать m равным 28 или 216. В этом случае хеш-функция будет равняться одной или двум последним буквам ключа. Для предотвращения скучивания ключей следует выбирать m равным простому числу.

Метод деления часто используется после применения другой хеш-функции для соответствия полученных значений размеру хеш-таблицы.

Метод свертки (слияния). Предположим, что ключ представлен в виде последовательности разрядов a_i: $k = a_1\ a_2\ a_3\ a_p$, где p кратно некоторому числу w. Тогда значением хеш-функции будет сумма:

$$h(k) = a_1\ a_2\ a_w \oplus a_{w+1}\ a_{w+2}\ a_{2w} \oplus \ldots \oplus a_{p-w+1}\ a_{p-w}\ a_p\ ,$$

где в качестве операции $\oplus$ может использоваться операция арифметического или побитового сложения, побитовая операция "исключающее или" и т. д.

Для символьных ключей в качестве w удобно выбирать значения, кратные восьми. Основной недостаток этого метода состоит в том, что он не очень чувствителен к порядку символов. Но избавиться от этого просто. Допустим, что результат каждого последовательного применения операции $\oplus$ сохраняется в переменной h, конечное значение которой было результатом вычисления хеш-функции. Тогда для воздействия порядка символов на значение $h(k)$ необходимо применять операцию циклического сдвига h перед очередным применением операции $\oplus$.

Метод умножения. Представим значение ключа k в виде двоичного числа и примем размер хеш-таблицы m равным 2^P. Умножим дробь d на k и возьмем дробную часть числа, которую обозначим как $\{k\ d\}$, а в качестве значения хеш-функции используем p старших разрядов этой дробной части, т. е.

$$h(k) = \lfloor m\ \{k\ d\} \rfloor,$$

где $\lfloor x \rfloor$ - наибольшее целое число, не превосходящее x. Рекомендуется в качестве значения d брать иррациональное число, например золотое сечение $d = \frac{\sqrt{5}-1}{2}$. При $d = 1/m$ метод эквивалентен методу деления.

Метод "середины квадрата". Пусть m (размер хеш-таблицы) равен 2^P. Обозначим $d = k^2$ и представим d в виде двоичного числа. Тогда значением хеш-функции $h(k)$ будет p битов средней части d. Данный метод по многим параметрам уступает методу умножения.

Метод преобразования системы счисления. В основе метода лежит преобразование значения ключа k, выраженного в системе счисления с основанием p $(k = a_0\ p^0 + a_1\ p^1 + a_2\ p^2 + ...)$, в систему счисления с основанием q $(h(k) = a_0\ q^0 + a_1\ q^1 + a_2\ q^2 + ...)$ при условии, что $p < q$. Трудоемкость (число операций) этого метода оказывается большей, чем методов деления или умножения.

Метод деления многочленов. Пусть k, выраженное в двоичной системе счисления, записывается как $k = 2^n b_n + ... + 2b_1 + b_0$, и пусть размер хеш-таблицы m является степенью двойки $m = 2^P$. Представим двоичный

ключ k в виде многочлена вида $k(t) = b_n t^n + ... + b_1 t + b_0$. Определим остаток от деления этого многочлена на постоянный многочлен вида

$$c(t) = t^m + c_{m-1} t^{m-1} + ... c_1 t + c_0.$$

Этот остаток, рассматриваемый в двоичной системе счисления, используется в качестве значения хеш-функции $h(k)$. Для вычисления остатка от деления многочленов используют полиномиальную арифметику по модулю 2. Если в качестве $c(t)$ выбрать простой неприводимый многочлен, то при условии близких, но не равных $k1$ и $k2$, обязательно будет выполняться условие $h(k1) \neq h(k2)$. Многочлен $c(t)$ называется простым неприводимым многочленом, если его нельзя представить в виде произведения $c(t) = q(t) \cdot r(t)$, где $q(t)$ и $r(t)$ — многочлены, отличные от константы. Эта функция обладает сильным свойством рассеивания скученностей.

5.4.2. Методы разрешения коллизий

При работе с хеш-таблицей выделяются три основные операции: вставка, поиск и удаление элемента. Причем существует круг задач, в которых используются только первые две. При решении задачи следует учитывать набор предполагаемых операций, так как, например, операция удаления может привести к изменению структуры данных, соответствующей используемому методу разрешения коллизий хеш-адресов.

Методы разрешения коллизий можно разделить на два класса: метод цепочек и метод открытой адресации.

Метод цепочек. В этом методе в каждую запись хэш-таблицы добавляется указатель для поддержания связанного списка. Сами списки могут размещаться как в памяти, принадлежащей хеш-таблице (внутренние цепочки), так и в отдельной памяти (внешние цепочки).

При использовании внешних цепочек хеш-таблица представляет собой массив m связанных списков, называемых сегментами (см. рис. 5.21). После вычисления значения хеш-функции $a = h(k)$ задача сводится к последовательному поиску, вставке или удалению элемента в a-ом списке.

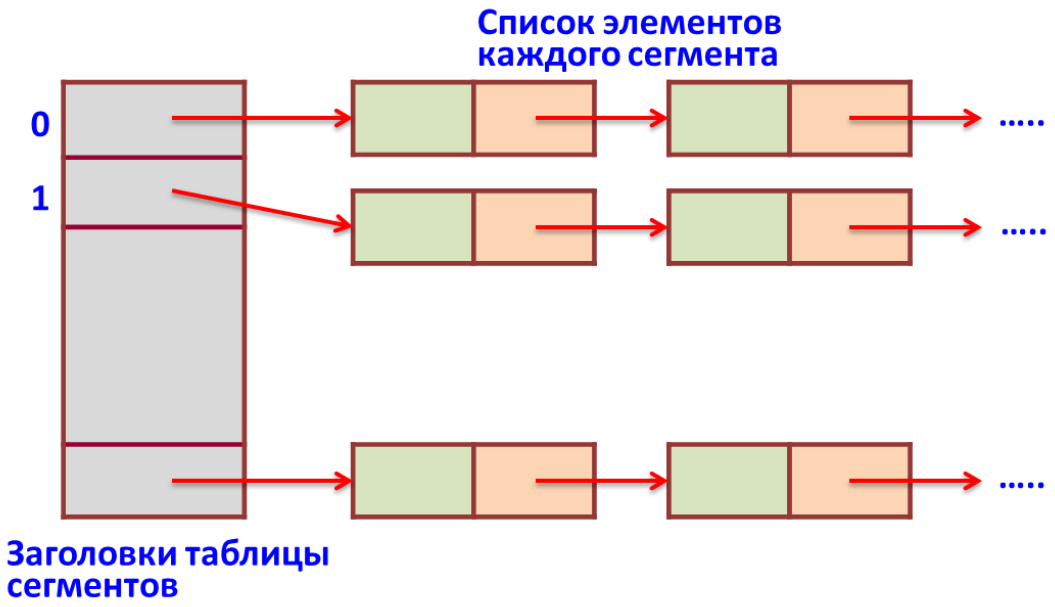


Рис. 5.21.Структура данных для представления хеш-таблицы с помощью внешних цепочек

Метод открытой адресации. В этом методе все элементы хеш-таблицы хранятся в одномерном массиве. Если при добавлении нового элемента возникает коллизия, то производится поиск свободного места в следующей ячейке таблицы. Адрес следующей ячейки вычисляется при помощи некоторой функции, аргументами которой в общем случае являются: значение ключа, первичный хеш-адрес, полученный при первом применении хеш-функции к текущему ключу, номер шага при поиске свободной ячейки.

Введем обозначения: m – размер хеш-таблицы, k – значение ключа; $h_0(k)$ – первичный хеш-адрес; i – номер шага при поиске свободной ячейки, $i = 1,2, ...$; $h_i(k)$ – значение хеш-адреса, полученного на i-ом шаге. Тогда алгоритм поиска/добавления элемента будет выглядеть так.

1. Полагаем $i = 1$.

2. Вычисляем $a = h_i(k)$.

3. Если a свободно, то алгоритм завершается (процедура вставки сохраняет элемент в ячейке a, процедура поиска сообщает об отсутствии ключа в хеш-таблице).

4. Если ключ в ячейке *a* равен *k*, то алгоритм завершается (процедура вставки повторно не сохраняет элемент в ячейке, процедура поиска сообщает о найденном элементе).

5. Обнаружена коллизия. Полагаем $i = i + 1$ и переходим к пункту 2.

Для вычисления первичного хеш-адреса $h_0(k)$ может использоваться любая из рассмотренных ранее функций расстановки. Значения хеш-адреса для *i*-го шага вычисляется по формуле $h_i(k) = (h_0(k) + c\ i) \bmod m$, где *c* - константа. В простейшем случае *c* полагается равной 1 или -1. В этом случае возникает опасность скучивания синонимов. Для устранения скучивания *c* и *m* должны быть взаимно простыми, а *c* – не слишком малым числом.

В методах разрешения коллизий, за исключением метода, основанного на хеш-таблицах с внешними цепочками, удаление элемента из хеш-таблицы не всегда является операцией обратной вставке, поскольку при удалении нарушается связь между синонимами. Чтобы этого не произошло, можно, не производя удаления записи, вместо ключа, ставшего ненужным, вписать специальный код. Этот код во время поиска пропускается, но разрешает использовать эту позицию для вставки нового элемента. Таким образом, каждая позиция хеш-таблицы может находиться в одном из трех состояний: свободном, занятом или удаленном. Если не предпринимать мер по переводу позиций из состояния "удалено" в состояние "свободно", то при постоянном изменении хеш-таблицы может наступить ситуация, когда не останется свободных ячеек. Скорость поиска заметно ухудшится и будет соответствовать линейному поиску элемента в неупорядоченной последовательности.

Вопросы для повторения темы 5

1. Дайте определение типу данных "Список".
2. Какие виды списков вы знаете? В чем особенности их структурной организации?

3. Реализация списков посредством массивов. Какие достоинства и недостатки у этого подхода?
4. Реализация списков с помощью указателей. Какие достоинства и недостатки у этого подхода?
5. Дайте определение типу данных "Стек". Какие операции предусматривает этот тип данных?
6. Для каких целей обычно применяются стеки?
7. Дайте понятие типу данных "Очередь". Какие операции предусматривает этот тип данных?
8. Для каких целей обычно применяются очереди?
9. Что такое очередь приоритетов? Каковы особенности ее реализации?
10. Что такое хеш-таблица? Для каких целей она применяется?
11. Какие функции расстановки, применяемые в хеш-таблицах вам известны?
12. Что такое коллизия в хеш-таблицах? Какие методы разрешения коллизий вы знаете?